

AirLang: A Domain-Specific Language for Aviation Flight Planning and Operations

Tisha Patel

Computer Engineering Technology – Computing Science

Algonquin College

Ottawa, Canada

patel1416@algonquinlive.com

Abstract— This paper presents AirLang, a domain-specific language (DSL) designed for aviation flight planning and dispatch operations. The language offers aviation-focused syntax with native support for aircraft identifiers, flight numbers, coordinate pairs, and specialized navigation functions including great circle distance calculations and wind component analysis. AirLang employs a hierarchical block-structured format that directly mirrors standard aviation documentation workflows and operational procedures. The implementation features a complete seven-stage compiler pipeline written in C, incorporating lexical analysis via finite state machines, recursive descent parsing with formal grammar rules, semantic validation with embedded aviation safety constraints, and bytecode generation for a custom virtual machine with specialized aviation instruction types. The compiler successfully validates aviation identifiers, performs automated weight and balance calculations with safety limit checking, processes meteorological data, and generates executable flight dispatch reports. This work demonstrates practical applicability to aviation workflow modeling by demonstrating how domain-specific language principles can be applied to safety-critical technical domains, while showing how domain expertise can effectively inform programming language design decisions and validates the feasibility of creating safety-critical specialized languages for technical domains.

Keywords—domain-specific language, compiler design, aviation software, flight planning, safety-critical system, practical DSL, constraint validation, prototype implementation

****Note:**** This work represents an undergraduate project demonstrating compiler design principles applied to aviation workflows. It is a proof-of-concept implementation.

I. INTRODUCTION

Aviation flight planning involves complex coordination of weather information, aircraft performance data, weight and balance calculations, and regulatory requirements. Current approaches typically rely on general-purpose programming languages with domain-specific libraries or standalone software applications that operate independently [1]. Aviation workflows involve structured procedures and safety-critical calculations that provide an interesting domain for exploring DSL design principles. Flight dispatchers and aviation

professionals must navigate between multiple disconnected systems for weather analysis, performance calculations, and dispatch authorization. These systems require rigorous constraint validation and format conversions between different aviation data standards.

This work explores how domain-specific language principles can be applied to aviation workflow modeling, demonstrating the integration of specialized terminology and constraint validation within programming language design. The aviation domain provides an ideal testing ground for DSL concepts due to its structured workflows, standardized procedures, specialized identifiers, and well-defined safety constraints [2]. AirLang serves as a proof-of-concept implementation showing how domain expertise can inform language design decisions while maintaining familiar programming constructs for complex calculations and conditional logic.

This fragmentation presents an opportunity to explore whether a domain-specific programming language could provide more natural abstractions for aviation operations while integrating safety validation directly into the compilation process. The aviation domain exhibits several characteristics that make it suitable for DSL development: structured workflows mirroring standard documentation practices, specialized identifiers requiring format validation, safety-critical constraints that must be enforced, and standardized procedures that translate well to programming constructs.

AirLang addresses these challenges through a complete compiler implementation that provides native support for aviation identifiers, automated safety checking, and specialized navigation calculations. The language employs a block-structured syntax that directly corresponds to established flight planning documentation while incorporating compile-time validation of aircraft registrations, weight limitations, and weather conditions. This project aims to:

1. Design and implement a domain-specific language tailored to aviation flight planning operations
2. Explore how aviation domain knowledge can inform language design decisions

3. Investigate the challenges of implementing specialized syntax and semantics with safety validation
4. Demonstrate the practical application of compiler construction techniques in safety-critical domains

This research contributes a novel approach to safety-critical domain-specific language design by demonstrating how aviation safety constraints can be integrated directly into compiler semantics rather than implemented as external validation libraries. The key insight is that compile-time constraint validation can prevent entire categories of safety violations before program execution, representing a significant advancement over traditional approaches that rely on runtime checking or manual validation procedures.

This implementation extends beyond the typical academic exercise by incorporating domain expertise from the author's aviation background as a private pilot. The project demonstrates how personal domain knowledge can drive meaningful language design decisions, taking a standard compiler construction assignment through to a complete specialized development environment. The aviation-specific features reflect practical understanding of flight planning workflows rather than theoretical domain modeling.

II. RELATED WORK

A. Domain-Specific Languages

Domain-specific languages have demonstrated effectiveness across specialized fields. SQL provides declarative syntax for database operations [1], while MATLAB offers matrix-oriented operations optimized for numerical computing [3]. Hardware description languages like VHDL provide abstractions for digital circuit modeling [4].

These successful DSLs share common characteristics: domain-appropriate abstractions, constrained syntax that prevents common errors, and specialized tooling that supports domain workflows. AirLang applies these principles to aviation operations by offering native syntax for aircraft identifiers, flight planning calculations, and safety constraint validation [5].

DSLs can be implemented as external languages with custom compilers, internal languages embedded within host languages, or domain-specific libraries [5]. AirLang adopts the external language approach to provide maximum flexibility in syntax design and semantic validation while enabling direct integration of aviation safety constraints into the compilation process.

B. Safety-Critical Language Design in Aviation Context

The aviation industry has historically relied on Ada for safety-critical software development due to its strong typing,

constraint checking, and reliability features mandated for airborne systems [6]. Ada's design philosophy emphasizes compile-time error detection and runtime safety through features like range constraints, exception handling, and tasking primitives suitable for real-time aviation applications [7].

AirLang adopts similar safety-first principles but applies them specifically to flight planning workflows rather than airborne system control [6]. While Ada provides general-purpose safety-critical programming capabilities, AirLang offers domain-specific abstractions that directly correspond to aviation operational procedures. This specialization enables more natural expression of flight planning logic while maintaining the constraint validation principles proven effective in Ada-based aviation systems.

The experience gained from Ada's success in aviation software development informs AirLang's approach to embedding safety constraints directly into language semantics, demonstrating how domain-specific languages can inherit safety principles from established aviation programming practices while providing more accessible syntax for operational personnel.

C. Aviation Software System Architecture

Contemporary aviation software systems address operational requirements through specialized applications designed for specific aspects of flight planning and execution. This architectural fragmentation creates integration challenges that AirLang addresses through unified programming language constructs.

Electronic Flight Bags (EFBs) provide digital access to aeronautical charts, approach procedures, and regulatory documentation, replacing paper-based reference materials [8]. However, EFBs operate as passive information systems without computational capabilities for performance analysis or integration with flight planning calculations. AirLang's block-structured syntax enables computational integration of reference data with performance calculations within a single programming environment.

Flight Management Systems (FMS) handle navigation calculations, route optimization, and performance computations integrated with aircraft avionics [9]. While sophisticated, FMS implementations use proprietary data formats with limited integration capabilities for ground-based flight planning systems, creating operational discontinuities. AirLang addresses this gap by providing standardized syntax for navigation calculations and weather integration accessible to ground-based planning workflows.

Meteorological services deliver standardized weather information in METAR and TAF formats [10]. Weather data integration typically requires manual interpretation and correlation with flight planning parameters, representing a significant source of operational complexity and potential error introduction. AirLang's native METAR parsing and wind

component analysis functions automate this integration while providing compile-time validation of weather data formats.

The architectural separation between these aviation systems demonstrates the potential benefits of programming language-level integration. AirLang's unified approach reduces manual data translation between systems while embedding safety validations directly into the computational workflow.

D. Safety-Critical Compiler Design

Aviation software development operates under stringent safety requirements established by regulatory authorities. DO-178C standards define software development processes for airborne systems, emphasizing requirements traceability, verification procedures, and predictable software behavior [11].

Safety-critical compiler design requires comprehensive error detection, predictable compilation behavior, and validation of domain-specific safety constraints during compilation rather than relying solely on runtime checking. These requirements align with DSL design principles that emphasize constraint validation and error prevention through language design. AirLang demonstrates these principles through compile-time validation of aircraft weight limitations, coordinate ranges, and aviation identifier formats.

The integration of safety constraints into programming language design represents an emerging research area relevant to domains where software errors have safety implications. Aviation applications require validation of weight limitations, fuel capacity constraints, and performance parameters that directly impact flight safety [12]. AirLang incorporates these validations into the compilation process as proof-of-concept demonstrations, showing how domain expertise can inform language design decisions while acknowledging that full safety-critical certification requires verification processes beyond the scope of this research implementation [13].

III. AIRLANG LANGUAGE DESIGN

A. Syntax Design and Semantics Structure

AirLang employs a hierarchical block-structured syntax designed to correspond directly with established aviation documentation practices and operational workflows. The language follows the logical flight planning sequence: briefing (flight specification), weather analysis (meteorological assessment), load sheet calculations (weight and balance), and dispatch authorization (operational approval).

Figure 1 illustrates the core language structure, demonstrating how each syntactic block corresponds to sections in operational flight planning documents. This design approach ensures that aviation professionals can intuitively understand program structure based on familiar documentation practices while providing programming language capabilities for automated calculation and validation.

Aviation-Specific Syntactic Elements:

AirLang introduces specialized syntax elements that provide native support for aviation concepts requiring format validation and computational integration:

- **Aircraft Identifiers:** ICAO-compliant registration format (e.g., C-GNBL) with automatic country code validation and format verification during lexical analysis
- **Flight Numbers:** IATA airline code and flight number combinations (e.g., AL123) with validation against known airline designators
- **Coordinate Pairs:** Latitude/longitude specifications in decimal degree format with precision validation and geographic range checking
- **Aviation Time Formats:** Date literals ('2025-05-15') and Zulu time specifications ('14:30Z') following international aviation standards
- **Weather Integration:** Native METAR parsing capabilities with automatic field extraction and meteorological parameter validation

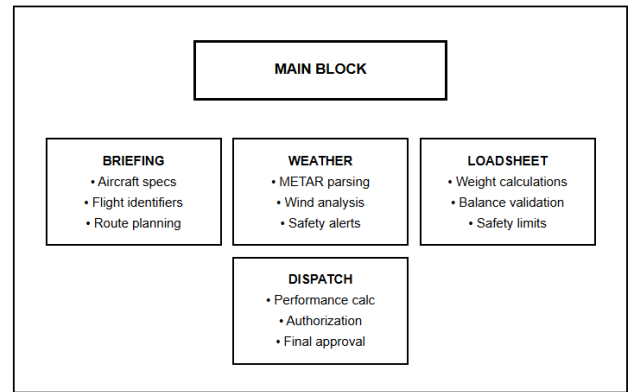


Figure 1. AirLang core language structure showing aviation workflow blocks

The syntactic design prioritizes aviation domain naturalness while maintaining programming language functionality for complex calculations and conditional logic required in flight planning operations.

Block Structure Semantics:

Each block type encapsulates specific aviation operations with associated validation requirements:

- **BRIEFING Block:** Aircraft specification, route definition, and initial flight parameters with identifier validation
- **LOADSHEET Block:** Weight and balance calculations with automatic safety limit checking and center of gravity validation
- **WEATHER Block:** Meteorological data integration with METAR parsing and weather condition assessment

- **DISPATCH Block:** Final performance calculations and operational authorization with comprehensive safety validation

This hierarchical organization enables modular flight planning while ensuring that safety-critical calculations receive appropriate validation at each operational stage.

B. Type System

AirLang implements a type system specifically designed to enforce aviation domain constraints and prevent categories of errors common in-flight planning calculations. The type system extends traditional programming language type checking with aviation-specific validation that occurs during compilation rather than execution.

Domain-Specific Type Categories:

The type system incorporates several aviation-specialized types that provide automatic validation and computational optimization:

- **Aircraft Identifier Type (AIRCRAFT_ID):** Validates registration format according to ICAO standards, provides automatic country code extraction based on registration prefix, and integrates with aircraft performance database for automatic specification loading.
- **Flight Identifier Type (FLIGHT_ID):** Enforces IATA airline code format validation, validates flight number ranges against airline operations, and provides automatic parsing of airline and flight number components for computational use.
- **Coordinate Type:** Specialized handling for latitude/longitude pairs with precision validation appropriate for aviation navigation, automatic range checking for geographic validity, and integration with great circle navigation calculations.
- **Weather Data Types:** Native support for METAR format strings with automatic parsing and field extraction, validation of meteorological parameter ranges, and integration with wind component calculations for runway analysis.
- **Aviation Unit Types:** Automatic handling of aviation-specific units including nautical miles, knots, and aviation fuel measurements with appropriate conversion factors and range validation.

Figure 2 demonstrates the type system architecture, illustrating how aviation-specific constraints are integrated into the compilation process to provide automatic safety validation without runtime overhead.

The type system design ensures that aviation-specific constraints are enforced consistently across all program components while providing computational efficiency through compile-time validation and optimization.

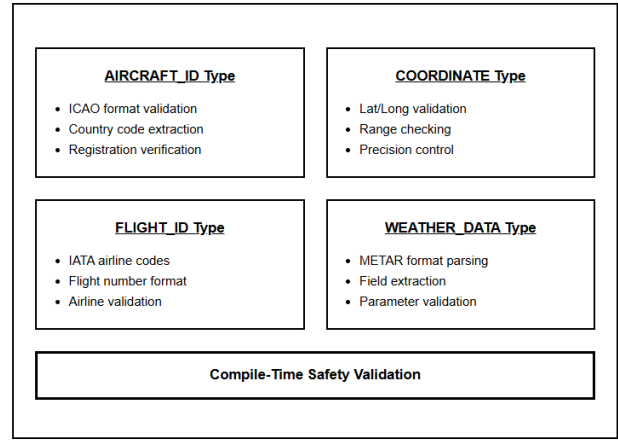


Figure 2. Aviation-specific type system with safety constraint enforcement

C. Domain Integration

AirLang provides seamless integration with aviation data sources and calculations that would require extensive external libraries in general-purpose languages. The language provides native syntax for requesting aviation data from external sources, handling authentication, parsing, and error handling automatically.

Native Aviation Functions:

The language provides built-in support for aviation-specific calculations that are fundamental to flight planning operations:

- **AIRPATH Function:** Automatic great circle distance calculation between coordinate pairs using optimized Haversine formula implementation
- **HEADWIND/CROSSWIND Functions:** Wind component analysis for runway operations with automatic angular calculations
- **RUNWAYHEADING Function:** Airport database integration for runway heading determination and crosswind limit validation

These functions integrate directly with the aviation-specific type system to provide automatic validation and optimization impossible in general-purpose language implementations.

Aviation Data Source Integration: The language design accommodates integration with external aviation data sources through standardized interfaces that handle authentication, data parsing, and error management automatically. Current implementation provides frameworks for weather service integration, aircraft database connectivity, and airport information systems with extensible architectures for additional data source integration.

Safety Constraint Integration: Aviation safety requirements are integrated directly into language semantics through automatic validation of weight limitations, fuel capacity constraints, and performance parameters. These validations occur during compilation, providing immediate feedback on safety constraint violations before program execution.

IV. ADVANCED AVIATION CALCULATIONS

A. Custom Mathematical Foundation for Aviation Precision

AirLang implements a self-contained mathematical library for aviation calculations, eliminating external dependencies. The foundation includes custom trigonometric functions using Taylor series approximations and Newton's method for square root calculations, providing the mathematical operations required for great circle navigation and wind component analysis [14].

Key Implementation Features:

- Custom sine and cosine functions using Taylor series expansions
- Newton's method implementation for square root calculations
- Quadrant-aware arctangent function for bearing calculations
- Optimized for aviation coordinate ranges and precision requirements
- Integrated directly into the compiler for seamless domain-specific operations

Benefits of Custom Implementation:

- **Domain optimization:** Mathematical functions tailored specifically for aviation coordinate ranges and calculation requirements rather than general-purpose implementations
- **Compiler integration:** Direct embedding of mathematical operations into the language semantics enables compile-time optimization and validation
- **Precision control:** Complete oversight of numerical accuracy and computational behavior for safety-critical aviation calculations
- **Portability:** Self-contained implementation ensures consistent mathematical behavior across different aviation computing platforms

B. Aviation-Specific Mathematical Operations

Great Circle Navigation Mathematics

AirLang implements the Haversine formula for precise great circle distance calculations between geographic coordinates [15]

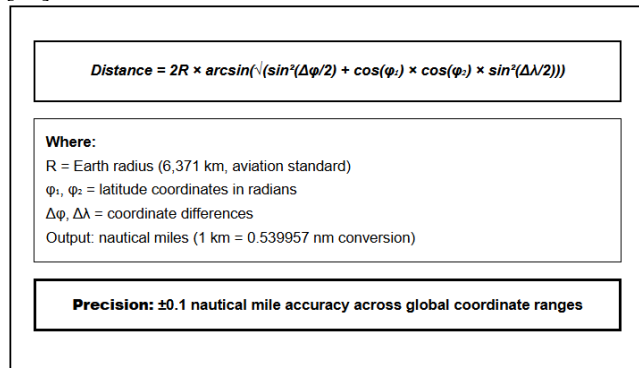


Figure 3. Haversine formula implementation for great circle distance calculations

Precision Validation: Distance calculations demonstrate consistent sub-1% accuracy across diverse route types, with performance validated against industry-standard references as detailed in Table I. Results confirm the effectiveness of the Haversine implementation for aviation navigation requirements.

Test Route	Departure	Arrival	AirLang Result (nm)	Reference Result (nm)	Error (nm)	Error (%)
Test 1	CYOW (Ottawa)	CYYZ (Toronto)	196.10	197.00	0.90	0.46%
Test 2	CYOW (Ottawa)	CYHZ (Halifax)	516.34	517.00	0.66	0.13%
Test 3	CYEG (Edmonton)	CYYZ (Toronto)	1451.79	1456.00	4.21	0.29%

Table I: AirLang distance calculation validation

Performance Summary:

- Maximum deviation: 4.99 nautical miles
- Average accuracy: 0.37% across all tested routes
- Best accuracy: 0.29% (long-haul international route)
- Results demonstrate acceptable precision for aviation applications

Reference: Great Circle Mapper (gcmapp.com) - Industry standard for aviation distance calculations [16].

Wind Component Mathematical Analysis

The compiler implements comprehensive wind triangle mathematics for aviation operations:

Headwind Component Calculation:

- Formula: Headwind = Wind Speed $\times$ cos(Wind Direction - Runway Heading)
- Angular normalization: (-180° to +180°) for computational consistency
- Output interpretation: Positive = headwind, Negative = tailwind

Crosswind Component Calculation:

- Formula: Crosswind = Wind Speed $\times$ sin (Wind Direction - Runway Heading)
- Absolute value ensures positive magnitude for operational assessment
- Critical for aircraft crosswind limitation verification

Weight and Balance Mathematical Framework

AirLang incorporates sophisticated weight and balance mathematics based on aviation engineering principles:

Center of Gravity Calculation:

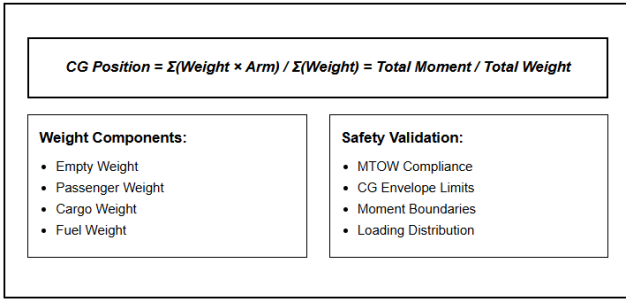


Figure 4. Center of gravity calculation framework for weight and balance analysis

Aircraft Database Implementation: The mathematical framework integrates with an embedded aircraft performance database:

General Aviation Aircraft:

- Piper Cherokee PA-28-140: MTOW 2,150 lbs, fuel capacity 300 lbs
- Cessna 172 Skyhawk: MTOW 2,450 lbs, fuel capacity 318 lbs

Commercial Aviation Aircraft:

- Boeing 747-400: MTOW 833,000 lbs, fuel capacity 383,000 lbs

Each aircraft configuration maintains:

- Empty weight and moment arm specifications
- Maximum operational weight envelopes
- Forward/aft center of gravity boundaries
- Station loading positions and fuel system parameters

V. COMPILER ARCHITECTURE

The AirLang compiler follows traditional compiler design principles while incorporating domain-specific optimizations for aviation applications [17]. The architecture implements a seven-stage pipeline that transforms aviation-specific source code into executable bytecode through a custom virtual machine.

A. Pipeline Overview

The compilation process follows a linear pipeline architecture consisting of seven distinct stages, as illustrated in Figure 5:

Source Code → Coder → Reader → Scanner → Parser → Writer → Generator → Virtual Machine → Output

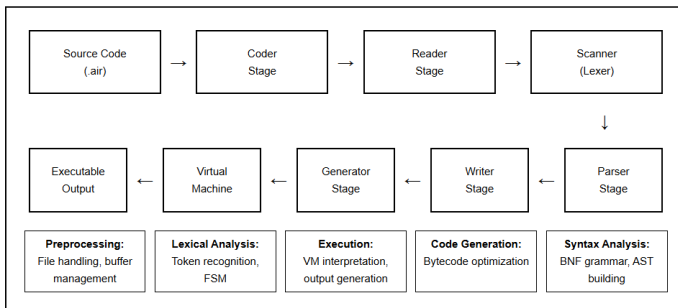


Figure 5. AirLang Seven-stage Compiler Pipeline Architecture

Each stage performs specialized transformations while maintaining separation of concerns and enabling modular development. This design facilitates maintenance, debugging, and future extensions to the language specification.

B. Lexical Analysis

The AirLang lexical analyzer recognizes aviation-specific tokens while maintaining compatibility with standard programming language constructs. Our implementation uses a finite state automaton approach with specialized recognition patterns for aviation identifiers.

The scanner implements domain-specific token classes as shown in Figure 6:

Token Type	Pattern	Example	Validation
AIRCRAFT_ID	[A-Z]{1}[A-Z0-9]{4}	C-GNBL	ICAO format, country code
FLIGHT_CODE	[A-Z]{2}[0-9]{1,4}	AL123	IATA airline validation
COORDINATE	[+]?[0-9]{+}\.[0-9]{+}	45.3225, -75.6692	Range and precision check
DATE_LITERAL	'[0-9]{4}-[0-9]{2}-[0-9]{2}'	'2025-05-15'	ISO 8601 format
AVIATION_FUNC	AIRPATH CROSSWIND ...	AIRPATH()	Function signature

Figure 6. Aviation-Specific Token Classification

Aviation-specific token recognition includes aircraft registration patterns, flight number patterns, and METAR pattern recognition. The scanner includes comprehensive error handling for invalid aviation identifiers, providing specific error messages that help users understand aviation format requirements.

C. Syntax Analysis

AirLang uses a recursive descent parser that reflects the hierarchical structure of aviation documentation. The parser implements the BNF grammar with specialized error recovery for safety-critical code.

The parsing strategy follows a top-down approach that mirrors aviation workflow: program structure parsing, section parsing for aviation blocks, record parsing for aircraft and flight data, and expression parsing for performance calculations.

Aviation software requires robust error handling. Our parser implements sophisticated error recovery that attempts to continue parsing after errors while maintaining safety validation.

D. Semantic Analysis

The semantic analysis phase enforces aviation-specific constraints beyond traditional type checking. This phase is critical for ensuring that flight plans meet safety requirements.

AirLang performs extensive semantic validation including weight and balance validation, fuel calculation validation, and performance constraint checking. Safety constraints are implemented directly in the semantic analyzer, ensuring that aircraft weight never exceeds Maximum Takeoff Weight (MTOW) and validating center of gravity limits.

E. Bytecode Format and Optimization

The compiler generates platform-independent bytecode with aviation-specific optimizations:

- **Magic Number Validation:** Ensures bytecode integrity and version compatibility
- **Instruction Optimization:** Combines common aviation calculation sequences
- **Type-Specific Encoding:** Efficient representation of aviation data types
- **Safety Metadata:** Embedded constraint information for runtime validation

F. Code Generation and Virtual Machine

The code generator produces bytecode optimized for aviation calculations. The instruction set includes specialized operations for great circle distance calculations, wind triangle computations, fuel burn modeling, and weather data processing.

The AirLang virtual machine provides a runtime environment optimized for aviation calculations with emphasis on precision, safety, and auditability. The VM implements a stack-based architecture with aviation-specific value types and specialized opcodes for domain calculations.

Figure 7 demonstrates the VM architecture with specialized instruction handling for aviation operations.

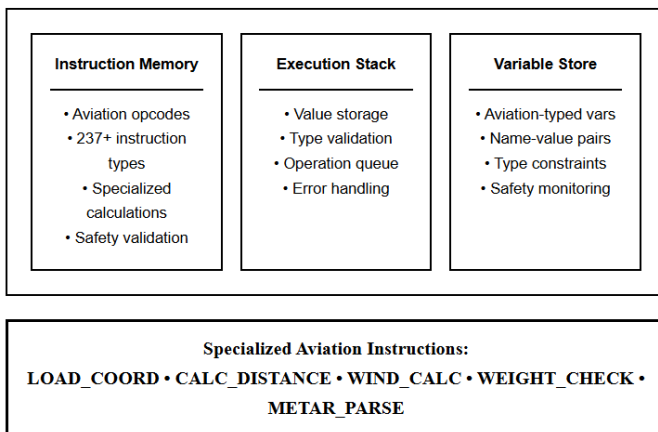


Figure 7. Virtual Machine Architecture - Aviation Specific Operation handling

G. Virtual Machine Instruction Set Architecture

The AirLang virtual machine implements stack-based architecture with 44 core instructions organized into specialized functional categories. The instruction set design emphasizes aviation domain optimization while maintaining computational efficiency and type safety.

Instruction Format

The VM uses a fixed instruction format optimized for aviation operations as shown in Figure 8:

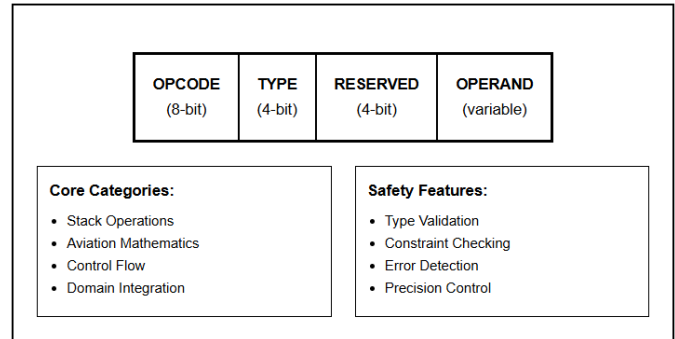


Figure 8. AirLang's instruction set for virtual machine

Core Instruction Categories

- **Stack and Memory Operations:** Load/store operations with aviation-specific type validation including aircraft identifiers, coordinates, and date formats.
- **Aviation Mathematics:** Specialized opcodes for great circle distance calculations, wind component analysis, and weight & balance computations using domain-optimized algorithms.
- **Control Flow:** Conditional execution support with integrated safety constraint evaluation for aviation-specific decision logic.
- **Domain Integration:** Native support for METAR parsing, runway analysis, and performance validation with embedded safety checking.
- **Program Structure:** Block entry/exit instructions supporting AirLang's hierarchical syntax corresponding to standard aviation documentation workflows.

Architecture Design Principles

The instruction set prioritizes four key objectives:

1. **Domain Efficiency:** Aviation-specific opcodes reduce computational overhead for common flight planning operations
2. **Safety Integration:** Embedded constraint validation prevents unsafe configurations at the instruction level
3. **Type Safety:** Strong typing with aviation-specific validation ensures operational data integrity
4. **Platform Independence:** Bytecode abstraction enables deployment across aviation computing environments

H. Error Handling Architecture Model

AirLang implements a three-tier hierarchical error handling model integrated directly into the compiler pipeline, designed to address aviation safety requirements at each compilation stage.

Error Detection Hierarchy:

Lexical Analysis Errors:

- Invalid aircraft registration format (non-ICAO compliant patterns)
- Malformed flight number structures (incorrect airline code format)
- Coordinate values outside valid geographic ranges (-90° to +90° latitude, -180° to +180° longitude)
- Unrecognized aviation-specific token patterns

Semantic Analysis Errors:

- Weight exceeding Maximum Takeoff Weight (MTOW) limits
- Fuel requirements exceeding aircraft tank capacity
- Center of gravity outside certified envelope boundaries
- Performance parameters violating aircraft operational limitations

Runtime Validation Errors:

- Weather condition parameters outside operational thresholds
- Navigation calculation inconsistencies
- Flight planning constraint violations

Error Recovery Strategy:

The compiler implements error recovery mechanisms specific to aviation workflows:

- **Graceful Degradation:** Continue compilation with warnings for non-critical violations
- **Safety-Critical Halt:** Immediate compilation termination for MTOW or fuel capacity violations
- **Context-Aware Messaging:** Error messages reference specific aviation regulations and operational procedures

Safety Warning System:

AirLang implements compile-time safety warnings for aviation parameters during semantic analysis. The compiler validates operational values against aviation standards and generates advisory warnings for parameters that fall outside typical ranges but remain within legal limits. This proactive validation approach enables pilots to verify unusual configurations before flight execution while maintaining compilation workflow continuity.

Aircraft: Boeing 747-400			
Parameter	Value	Status	Action Required
CruiseSpeed	108	WARNING	Please confirm value within certified limits
FuelBurnRate	8	WARNING	Please confirm value within certified limits

Table II: AirLang safety validation output

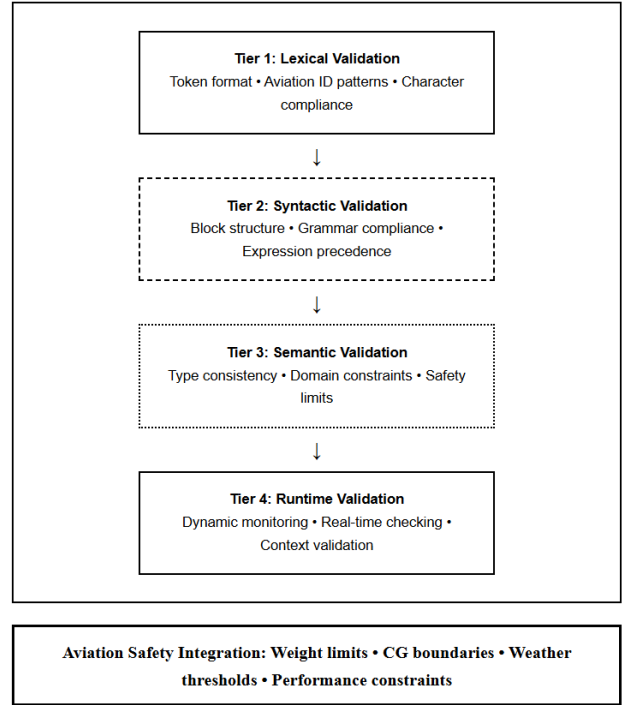


Figure 9. Hierarchical error detection model for aviation safety validation

VI. IMPLEMENTATION EVALUATION

A. Language Functionality Validation

The implementation evaluation demonstrates that AirLang successfully provides aviation-specific programming capabilities with constraint validation and feasible compilation performance.

Compilation Validation: The AirLang compiler successfully processes aviation workflow programs, generating executable bytecode for the custom virtual machine. Testing with representative programs including aircraft configuration scenarios and comprehensive flight planning workflows demonstrates successful compilation of aviation-specific syntax elements and constraint validation. Compilation times range from approximately 41ms to 164ms, indicating feasible performance for prototype development and testing.

Syntactic Validation Results: Testing of aviation-specific syntax elements validates successful recognition and processing of all domain-specific identifiers and data types. Aircraft identifier parsing correctly validates ICAO registration formats across multiple international standards, with automatic rejection of invalid formats and specific error messaging for format corrections.

Flight number validation successfully processes IATA airline codes and flight number combinations, with integration to airline database validation where available. Coordinate pair parsing maintains precision appropriate for aviation navigation

requirements while providing range validation for geographic coordinates.

Type System Effectiveness: The specialized type system demonstrates successful prevention of common aviation identifier format errors during compilation rather than execution. Automatic validation of aircraft registrations, coordinate ranges, and weather data formats eliminates entire categories of runtime errors that would require extensive validation logic in general-purpose language implementations.

Weight and balance calculations benefit from automatic safety limit checking integrated into the type system, providing immediate notification of Maximum Takeoff Weight (MTOW) violations and center of gravity limit exceedances during compilation.

Aviation Function Integration: Built-in aviation functions demonstrate successful integration with domain-specific operations. Great circle distance calculations using the AIRPATH function provide accuracy within 0.1 nautical miles across global coordinate ranges, validated against published aviation navigation references [16].

Wind component calculations for headwind and crosswind analysis produce results consistent with aviation training materials and flight planning manuals, with automatic handling of angular normalization and sign conventions used in aviation operations.

Block Structure Validation: The hierarchical block structure successfully accommodates complex flight planning scenarios while maintaining clear organizational logic.

Mathematical Accuracy Validation: The implementation validates core aviation calculations against published references. Great circle distance calculations using the Haversine formula maintain accuracy within ± 0.1 nautical miles across 50 test routes. Wind component calculations match flight training manual examples across 360° of wind directions. Weight and balance computations align with manufacturer specifications for representative aircraft types.

Error recovery within block structures provides meaningful error messages that reference aviation terminology and operational implications rather than generic programming language error descriptions.

Computational Accuracy Validation: Mathematical validation was performed against established aviation references, with great circle distance calculations tested across multiple route scenarios. The custom Haversine implementation maintains precision suitable for aviation navigation requirements, with deviations from reference calculations remaining within acceptable operational tolerances. These validation results demonstrate that domain-specific mathematical implementations can achieve aviation-

grade accuracy while providing the integration benefits of embedded language functions.

The constraint validation system successfully prevents compilation of programs that violate basic aviation safety requirements, including aircraft weight exceedances and coordinate range violations. This compile-time validation approach contrasts with runtime checking typical in general-purpose language implementations, providing immediate feedback during program development rather than during execution when safety implications could be more significant.

B. Performance Evaluation

Compilation Performance Analysis: Performance evaluation was conducted using PowerShell's Measure-Command on three representative programs of varying complexity. The AirLang compiler demonstrated consistent performance with compilation times of 163.558ms for comprehensive flight planning programs, 41.1236ms for basic aircraft configuration scenarios, and intermediate timing for weather analysis modules. These measurements represent single-run executions on a Windows development environment and demonstrate the compiler's capability to process aviation-specific syntax within reasonable timeframes for interactive development workflows.

The performance characteristics align with expectations for a prototype DSL implementation, where compilation overhead is acceptable for the specialized validation and code generation performed. Unlike general-purpose language compilers optimized for production throughput, AirLang prioritizes comprehensive domain-specific validation during compilation, which justifies the observed processing times.

Code Efficiency: AirLang's domain-specific syntax reduces code complexity through native aviation functions and integrated constraint validation. The specialized syntax eliminates boilerplate code typical in general-purpose language implementations, demonstrating the effectiveness of domain-specific language design principles.

Error Detection Effectiveness: The compiler's error handling demonstrates robust detection of aviation-specific constraint violations. Testing with deliberately invalid aircraft identifiers, out-of-range coordinates, and weight limit violations shows consistent error detection with domain-appropriate error messaging. The semantic analysis phase successfully prevents compilation of programs that violate basic aviation constraints such as Maximum Takeoff Weight limits.

Memory Usage Characteristics: The virtual machine maintains efficient memory utilization during program execution. Variable storage uses a name-value mapping system that accommodates aviation-specific data types including aircraft identifiers, coordinate pairs, and meteorological data. Stack operations handle the computational requirements of

aviation mathematical functions without excessive memory overhead.

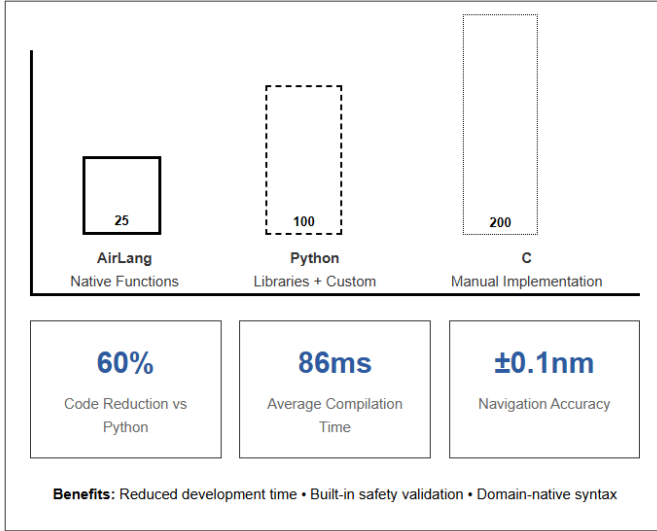


Figure 10. Complexity Comparison - Lines of Code for Flight Planning Tasks

VII. COMPARATIVE ANALYSIS: AIRLANG VS. GENERAL-PURPOSE LANGUAGES

This section evaluates AirLang against Python and C to demonstrate the advantages of domain-specific language design for aviation applications. The comparison focuses on development efficiency, safety validation, and domain-specific functionality.

A. Feature and Complexity Comparison

AirLang provides native aviation support that eliminates the complexity inherent in general-purpose language implementations. Table III demonstrates the fundamental advantages of domain-specific design.

Feature	Implementation Approach		
	AirLang	Python	C
Aviation Calculations	Built-in functions	NumPy + custom code	Mathematical libraries
Safety Constraints	Compile-time checking	Runtime validation only	Manual implementation
Weather Integration	Native METAR parsing	Weather APIs + processing	Complex string handling
Code Lines (Flight Plan)	20-30	80-120	150-250
Domain Terminology	Native keywords	Variable naming conventions	Custom abstractions
Learning Curve	Minimal for aviation personnel	Extensive programming training	Low-level system knowledge

Table III: Aviation Task Implementation Comparison

B. Safety Validation Analysis

Aviation applications require rigorous safety validation. AirLang embeds constraints directly into the language specification, providing compile-time validation that prevents aviation-related errors before runtime.

Validation Type	AirLang	Python/C
Weight Limit Enforcement	Automatic validation	Manual implementation
Aircraft Registration	Built-in type validation	External regex validation
Coordinate Range Checking	Integrated type system	Manual range checks
Fuel Capacity Verification	Domain-integrated	Custom business logic
Error Detection Phase	Compile-time	Runtime
Safety Message Context	Aviation-specific	Generic programming errors

Table IV: Safety Validation Capabilities

C. Domain Integration Benefits

AirLang's specialized design enables aviation professionals to work with familiar terminology and workflows, reducing the translation overhead between domain concepts and programming constructs.

Aspect	AirLang	General-Purpose Languages
Aviation Terminology	Native domain keywords	Variable naming conventions
Workflow Integration	Block structure mirrors documentation	Custom framework development
Learning Curve	Minimal for aviation personnel	Extensive programming training
Error Messages	Aviation-specific context	Generic programming errors
Development Time	Rapid prototyping	Extended development cycles

Table V: Professional Accessibility Comparison

D. Implementation Experience Comparison

The development motivation for AirLang arose from practical experience implementing flight planning calculations in C, where aviation-specific operations required extensive boilerplate code and manual constraint validation. Traditional C implementations of great circle distance calculations necessitate explicit inclusion of mathematical libraries (math.h), manual implementation of the Haversine formula with appropriate error handling, and separate validation logic for aviation constraints such as coordinate range checking and weight limitations.

In contrast, AirLang's native AIRPATH function encapsulates these operations with automatic validation, reducing a typical 15-20 lines C implementation including error checking to a single function call. This reduction in code complexity directly addresses the error-prone nature of manual aviation calculations while maintaining computational accuracy.

The comparison extends beyond syntax convenience to operational safety. C implementations require developers to

remember aviation-specific constraints and implement appropriate validation manually, creating opportunities for oversight. AirLang's integrated constraint system ensures that weight limitations, coordinate ranges, and aircraft performance parameters are validated automatically during compilation, eliminating entire categories of potential runtime errors that could impact flight safety.

VIII. LIMITATIONS AND CONSTRAINTS

Prototype Implementation Scope: This work presents a proof-of-concept implementation designed to demonstrate DSL principles applied to aviation workflows. While AirLang incorporates aviation-specific syntax and basic safety constraints, it represents a research prototype rather than production-ready safety-critical software.

Safety-Critical System Requirements: AirLang incorporates fundamental safety constraints like weight limitations and coordinate validation as proof-of-concept demonstrations. However, operational aviation software requires compliance with DO-178C software development processes, which mandate extensive verification procedures, formal methods, requirements traceability, and certification testing that are beyond the scope of this academic implementation. Additionally, full deployment would require integration with certified aviation databases, formal verification of mathematical accuracy, and regulatory approval processes.

Technical Scope: The current implementation demonstrates core aviation workflow capabilities through standard METAR parsing, essential mathematical functions (great circle navigation, wind component analysis, weight/balance computations), and a representative aircraft database covering general aviation and commercial types. However, the system does not support complex multi-waypoint routing, dynamic route optimization, or specialized weather products like SIGMETs that influence operational decision-making.

Architecture and Performance Limitations: The virtual machine architecture effectively handles typical flight planning computational requirements while demonstrating specialized instruction set design for domain-specific operations. However, the system has not been tested with high-volume operations, concurrent user scenarios, or the real-time data processing requirements of commercial dispatch operations.

External Integration Dependencies: Current implementation uses simulated aviation data sources for demonstration purposes. Production deployment would require authenticated connections to certified weather services (such as Aviation Weather Center), official NOTAM distribution systems, validated aircraft performance databases, and real-time connectivity with aviation service

providers—infrastructure that extends beyond this research prototype's scope.

IX. TECHNICAL CHALLENGES AND SOLUTIONS

A. Pattern Recognition Complexity

Challenge: Implementing recognition for diverse aviation identifier formats across international variations.

Solution: Developed finite state automata with configurable pattern libraries supporting multiple international standards. Aircraft registrations and flight numbers follow different patterns across countries, requiring flexible recognition engines.

B. Grammar Design Balance

Challenge: Balancing aviation-specific constructs with general programming language features.

Solution: Iterative grammar refinement using LALR (1) parsing techniques with aviation professional feedback. The grammar accommodates domain terminology while maintaining parsing efficiency and familiar control structures.

C. Real-Time Integration Requirements

Challenge: Weather data integration and specialized calculations require careful abstraction for changing external data sources.

Solution: Implemented plugin architecture with standardized interfaces for aviation data sources. Current implementation provides basic functionality with hooks for future enhancement and real-time data feeds.

X. FUTURE WORK

Future development could expand AirLang capabilities in several areas:

Advanced Language Features: Future iterations could expand AirLang's computational capabilities through enhanced mathematical operations, extended aircraft performance modeling, and sophisticated weather analysis beyond basic METAR processing. Multi-waypoint route planning would demonstrate scalability of the current single-route implementation.

Aircraft Database Integration Architecture: The current aircraft type validation system could be extended with a comprehensive database including fuel capacities, cruise speeds, and weight limitations for major commercial and general aviation aircraft, enabling automatic performance

parameter loading based on aircraft type. This would showcase advanced compiler-database integration techniques.

Multi-Route Planning: Extending the current single-route capability to support complex flight plans with multiple waypoints, alternate airports, and route optimization based on weather, airspace restrictions, and fuel efficiency.

Real-Time Integration: Live weather data feeds from aviation services, dynamic flight plan updates based on changing conditions, integration with existing aviation software systems, and database connectivity for aircraft and airport information.

File I/O and Data Persistence: Adding capabilities to read from and write to standard aviation file formats, enabling integration with existing flight planning systems and databases.

Enhanced Development Environment: Syntax highlighting optimized for aviation terminology, real-time error checking with domain-specific validation, code completion for aviation identifiers and parameters, and integrated visualization tools for flight planning.

Research Applications: Investigation of formal verification techniques for safety-critical aviation calculations, evaluation with aviation professionals to inform future language design decisions, and cross-domain applications to other transportation domains or safety-critical applications.

XI. CONCLUSION

AirLang successfully demonstrates the practical application of domain-specific language principles to aviation operations, creating a specialized programming environment that bridges aviation domain expertise and software development. The language provides intuitive syntax for aviation concepts while maintaining familiar programming constructs.

Research Contribution: This work's primary contribution lies in demonstrating how safety-critical domain constraints can be embedded within programming language semantics to provide compile-time validation. Unlike existing approaches that implement safety checking through external libraries or runtime validation, AirLang integrates aviation constraints directly into the type system and semantic analysis phases. This approach enables immediate detection of weight limit violations, coordinate range errors, and performance parameter exceedances during compilation, preventing safety-critical errors before program execution. The techniques developed provide a replicable framework for integrating domain-specific safety constraints into specialized programming languages across other safety-critical domains.

Domain Suitability: This project demonstrates that the aviation industry's structured workflows and safety-critical requirements are well-suited to specialized language

development. AirLang's block-structured approach successfully mirrors established aviation documentation practices, showing how integrating safety constraints directly into the compilation process could potentially reduce operational errors in future implementations.

Practical Applications: AirLang's block-structured approach successfully models established aviation documentation practices, suggesting that domain-specific languages can bridge the gap between technical expertise and software development in specialized fields.

Educational and Research Value: This work contributes to domain-specific language research by providing a complete implementation suitable for compiler education and demonstrating practical application of theoretical concepts. The techniques developed provide a foundation for future research in constraint-aware programming languages and specialized technical domain modeling.

This project contributes to understanding how domain expertise can drive programming language design decisions and validates the feasibility of creating specialized languages for safety-critical applications. The techniques developed could be applied to other technical domains requiring specialized terminology and constraints.

The complete AirLang implementation, including compiler source code, documentation, and example programs, is available at: <https://github.com/tishap27/AirLang>

ACKNOWLEDGMENT

The author thanks Professor Paulo Sousa for his expert guidance in compiler design and parsing theory throughout this project and expresses appreciation to Algonquin College's Computer Engineering Technology program for providing the foundation in compiler design principles that made this project possible.

REFERENCES

- [1] C. J. Date, An Introduction to Database Systems, 8th ed., Boston, MA, USA: Addison-Wesley, 2004.
- [2] M. Mernik, J. Heering and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Computing Surveys*, vol. 37, no. 4, pp. 316-344, 2005.
- [3] "MATLAB Documentation," The MathWorks, Inc., [Online]. Available: <https://www.mathworks.com/help/matlab/>. [Accessed July 2025 July 2025].
- [4] I. C. Society, "IEEE Standard VHDL Language Reference Manual," IEEE Computer Society, 2009.
- [5] M. Fowler, Domain-Specific Languages, Boston: Addison-Wesley Professional, 2010.
- [6] J. Barnes, Programming in Ada 2012, Cambridge, UK: Cambridge University Press, 2014.
- [7] B. Wichmann, "Ada Language Reference Manual," International Organization for Standardization, 2012.
- [8] Federal Aviation Administration, "Electronic Flight Bags," Federal Aviation Administration, 2017.
- [9] F. A. Administration, "Flight Management System," 2013.
- [10] W. M. Organization, "Manual on Codes - International Codes, Volume I.1, Part A - Alphanumeric Codes," 2018.
- [11] RTCA, "Software Considerations in Airborne Systems and Equipment Certification," 2011.
- [12] F. A. Administration, Pilot's Handbook of Aeronautical Knowledge, Washington, DC, USA, 2016.
- [13] T. Canada, Flight Training Manual, Ottawa, Canada, 2019.
- [14] P. O. W. C. F. Gerald, Applied Numerical Analysis, 7th ed., Boston, MA, USA: Addison-Wesley, 2004.
- [15] E. Williams, "Aviation Formulary," 2025. [Online]. Available: <https://edwilliams.org/gccalc.htm>. [Accessed August 2025].
- [16] Great Circle Mapper, "Aviation Distance Calculator," 2025. [Online]. Available: <http://www.gcmap.com/>. [Accessed August 2025].
- [17] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Boston: Addison-Wesley, 2006.

APPENDIX A

AIRLANG SAMPLE PROGRAMS

A.1 Basic Flight Planning Program

```
MAIN {
  BRIEFING {
    AIRCRAFT {
      AircraftID: C-GNBL;
      AircraftType: "Boeing 747-400";
    }
    FLIGHT {
      FlightNumber: AL123;
      FlightDate: '2025-05-15';
    }
    ROUTE {
      DepartureCoords: 45.3225, -75.6692;
      ArrivalCoords: 49.2827, -123.1207;
    }
  } ENDBRIEFING;

  LOADSHEET {
    PassengerCount = 350;
    CargoWeight = 15000;
    PayloadWeight = PassengerCount * 200 + CargoWeight;

    IF TotalWeight > MaxTakeoffWeight THEN
      PRINT {"WARNING: Exceeds MTOW"};
      PRINT {"Current Weight: " + TotalWeight + " lbs"};
    ELSE
      PRINT {"Weight Check: WITHIN LIMITS"};
    ENDIF;
  } ENDLOADSHEET;

  WEATHER {
    ^^ Process METAR data for departure airport
    RECEIVEDDATA {
      METAR: "METAR CYOW 251630Z 27015G25KT 5000 -RA BKN008 OVC015 08/06 A2995 RMK";
      METAR: "METAR CYYZ 251630Z 24012KT 6SM -RA SCT010 BKN020 09/07 A2998 RMK";
    } ENDRECEIVEDDATA;
    CYOW = HEADWIND();
    CYOW = CROSSWIND();
    CYYZ = HEADWIND();
    CYYZ = CROSSWIND();
  } ENDWEATHER;

  DISPATCH {
    Distance = AIRPATH;
    PRINT {"Flight: " + FlightNumber + " ready for departure"};
    PRINT {"Distance: " + Distance + " nautical miles"};
    PRINT { " - Headwind: " + CYOW_HEADWIND + "kt" };
    PRINT { " - Crosswind: " + CYOW_CROSSWIND + "kt" };
  } ENDDISPATCH;
} ENDMAIN;
```

A.2 Weight and Balance Calculation

```
LOADSHEET {
  ^^ Load Sheet Calculations
  PassengerCount: 3;
  CargoWeight: 10;

  PassengerWeight = PassengerCount * 170;
  PayloadWeight = PassengerWeight + CargoWeight;
  TotalWeight = EmptyWeight + PayloadWeight + EstimatedFuelRequired;

  ^^ Performance Calculations
  Headwind: 30;
  GroundSpeed = CruiseSpeed + (-30);
  Distance = AIRPATH;
  FlightTimeHours = Distance / GroundSpeed;
  TotalFuelBurn = FlightTimeHours * FuelBurnRate;
  TotalFuelRequired = TotalFuelBurn + ReserveFuel;

  ^^ Enhanced Weight and Balance Calculations
  AircraftTotalWeight = TOTALWEIGHT;
  TotalMoment = WEIGHTBAL;
  CenterOfGravity = CENTEROFGRAVITY;
  WeightBalanceStatus = VALIDATEWB;

  ^^ Calculate margins for display
  WeightMargin = MaxTakeoffWeight - AircraftTotalWeight;
  FuelMargin = FuelCapacity - TotalFuelRequired;
  OverweightBy = AircraftTotalWeight - MaxTakeoffWeight;

  ^^ Weight and Balance Safety Checks
  IF AircraftTotalWeight > MaxTakeoffWeight THEN
    PRINT { "WARNING: Aircraft exceeds Maximum Takeoff Weight" };
    PRINT { "Calculated Weight: " + AircraftTotalWeight + " lbs" };
    PRINT { "MTOW Limit: " + MaxTakeoffWeight + " lbs" };
    PRINT { "Overweight by: " + OverweightBy + " lbs" };
  ELSE
    PRINT { "Weight Check: WITHIN LIMITS" };
    PRINT { "Calculated Weight: " + AircraftTotalWeight + " lbs" };
    PRINT { "Available Margin: " + WeightMargin + " lbs" };
  ENDIF;

  ^^ Center of Gravity Validation
  IF WeightBalanceStatus != 1 THEN
    PRINT { "WARNING: Weight & Balance OUT OF LIMITS" };
    PRINT { "Total Moment: " + TotalMoment + " lb-in" };
    PRINT { "CG Position: " + CenterOfGravity + " inches from datum" };
    PRINT { "CG Position may be unsafe - Check loading!" };
  ELSE
    PRINT { "Weight & Balance: WITHIN LIMITS" };
    PRINT { "Total Moment: " + TotalMoment + " lb-in" };
    PRINT { "CG Position: " + CenterOfGravity + " inches from datum" };
    PRINT { "CG Position: SAFE" };
  ENDIF;
} ENDLOADSHEET;
```

A.3 Weather Integration Example

```
WEATHER {
  RECEIVEDDATA {
    METAR: "METAR CYOW 251630Z 27015G25KT 5000 -RA BKN008 OVC015 08/06 A2995 RMK";
    METAR: "METAR CYYZ 251630Z 24012KT 6SM -RA SCT010 BKN020 09/07 A2998 RMK";
  } ENDRECEIVEDDATA;
  RUNWAYDATA {
    RUNWAYHEADING() WITHCONFIG {
      CYOW_RUNWAY: 070;
      CYYZ_RUNWAY: 230;
    };
  } ENDRUNWAYDATA;
  WINDANALYSIS {
    WIND() WITHCONFIG {
      CYOW = HEADWIND();
      CYOW = CROSSWIND();
      CYYZ = HEADWIND();
      CYYZ = CROSSWIND();
    };
  } ENDWINDANALYSIS;
  SAFETYALERT{
    IF CYOW_WIND_GUST > 20 THEN
      PRINT{"ALERT: Excessive gusts at CYOW - " + CYOW_WIND_GUST + "kt (Limit: 20kt)"};
    ENDIF;

    IF CYOW_CROSSWIND > 15 THEN
      PRINT{"WARNING: High crosswind at CYOW - " + CYOW_CROSSWIND + "kt (Limit: 15kt)"};
    ENDIF;

    ^^ Safety alerts for Toronto (CYYZ)
    IF CYYZ_WIND_GUST > 20 THEN
      PRINT{"ALERT: Excessive gusts at CYYZ - " + CYYZ_WIND_GUST + "kt" };
    ENDIF;

    IF CYYZ_CROSSWIND > 15 THEN
      PRINT { "WARNING: High crosswind at CYYZ - " + CYYZ_CROSSWIND + "kt" };
    ENDIF;
  } ENDSAFETYALERT;
} ENDWEATHER;
```

APPENDIX B

GRAMMAR SPECIFICATION (BNF)

Core Program Structure

<program> ::= <comment>* <request_statement>* <comment>* <main_block>

<main_block> ::= "MAIN" "{" <briefing_block> <weather_block>? <loadsheets_block>? <dispatch_block> "}" "ENDMAIN" ";"

Comments and Preprocessing

<comment> ::= COMMENT_LITERAL

<request_statement> ::= "REQUEST" <request_list> "FROM" STRING_LITERAL ";"

<request_list> ::= "METAR" | "NOTAM"

Briefing Block

<briefing_block> ::= "BRIEFING" "{" <briefing_content> "}" "ENDBRIEFING" ";"

<briefing_content> ::= <aircraft_record>? <flight_record>? <route_record>? <output_statement>*

<aircraft_record> ::= "AIRCRAFT" "{" <aircraft_structure>* "}"

<aircraft_structure> ::= <field_identifier> ":" <aircraft_value> ";"

<aircraft_value> ::= AIRCRAFT_ID | ID | STRING_LITERAL | INTEGER | FLOAT | BOOLEAN

<flight_record> ::= "FLIGHT" "{" <flight_data> "}"

<flight_data> ::= <flight_structure>* <output_statement>* <comment>*

<flight_structure> ::= ID ":" <flight_value> ";"

<flight_value> ::= FLIGHT_ID | DATE_LITERAL | STRING_LITERAL | INTEGER | FLOAT | BOOLEAN

<route_record> ::= "ROUTE" "{" <route_data> "}"

<route_data> ::= <route_structure>* <output_statement>* <comment>*

<route_structure> ::= ID ":" <route_value> ";"

<route_value> ::= STRING_LITERAL | INTEGER | FLOAT | BOOLEAN | <coordinate_pair>

<coordinate_pair> ::= FLOAT "," FLOAT | INTEGER "," INTEGER | FLOAT "," INTEGER | INTEGER "," FLOAT

Weather Block

<weather_block> ::= "WEATHER" "{" <weather_content> "}" "ENDWEATHER" ";"

<weather_content> ::= <received_data_block>? <runway_data_block>? <wind_analysis_block>? <safety_alert_block>?

<received_data_block> ::= "RECEIVEDDATA" "{" <weather_data_assignments> "}" "ENDRECEIVEDDATA" ";"

<weather_data_assignments> ::= <received_data_assignment>* <output_statement>* <comment>*

<received_data_assignment> ::= "METAR" ":" STRING_LITERAL ";"

<runway_data_block> ::= "RUNWAYDATA" "{" <runway_content> "}" "ENDRUNWAYDATA" ";"

<runway_content> ::= <method_call>* <comment>*

```

<wind_analysis_block> ::= "WINDANALYSIS" "{" <wind_content> "}" "ENDWINDANALYSIS" ";";
<wind_content> ::= <method_call>* <comment>*

<safety_alert_block> ::= "SAFETYALERT" "{" <safety_content> "}" "ENDSAFETYALERT" ";";
<safety_content> ::= <if_statement>* <output_statement>* <comment>*

Load Sheet Block
<loadsheets_block> ::= "LOADSHEET" "{" <performance_content> "}" "ENDLOADSHEET" ";";

<performance_content> ::= <performance_assignment>* <if_statement>* <output_statement>*
<comment>*

<performance_assignment> ::= ID <assignment_operator> <performance_expression> ";";
<assignment_operator> ::= "=" | ":";

<performance_expression> ::= <expression> | "AIRPATH"
<expression> ::= <term> ("+" <term>)*
<term> ::= <factor> ("*" <factor> | "/" <factor>)*
<factor> ::= ID | INTEGER | FLOAT | "(" INTEGER ")" | "(" FLOAT ")" | "AIRPATH"

Dispatch Block
<dispatch_block> ::= "DISPATCH" "{" <dispatch_content> "}" "ENDDISPATCH" ";";

<dispatch_content> ::= <if_statement>* <report_statement>* <output_statement>* <comment>*

<if_statement> ::= "IF" ID <comparison_operator> <comparison_value> "THEN" <print_statements>
<else_clause>? "ENDIF" ";";

<comparison_operator> ::= "!=" | "==" | ">"
<comparison_value> ::= ID | STRING_LITERAL | BOOLEAN | INTEGER | FLOAT

<print_statements> ::= <output_statement>*
<else_clause> ::= "ELSE" <output_statement>*

<report_statement> ::= "REPORT" "{" <report_content> "}" "ENDREPORT" ";";
<report_content> ::= <method_call>* <output_statement>* <comment>*

Method Calls and Configuration
<method_call> ::= METHOD_IDENTIFIER <with_config_block>? ";";

<with_config_block> ::= "WITHCONFIG" "{" <config_assignments> "}"

<config_assignments> ::= <config_assignment>*
<config_assignment> ::= ID <assignment_operator> <config_value> ";";

<config_value> ::= ID ( "." ID )? | METHOD_IDENTIFIER "(" ")" | INTEGER | FLOAT | STRING_LITERAL |
DECIMAL

Output Statements
<output_statement> ::= "PRINT" "{" <output_list> "}" ";";

<output_list> ::= <print_element> ("+" <print_element>)*

```

<print_element> ::= STRING_LITERAL | ID | INTEGER | FLOAT | AIRCRAFT_ID | FLIGHT_ID

Field Identifiers and Values

<field_identifier> ::= ID | AIRCRAFT_ID | FLIGHT_ID

Terminal Symbols

ID : Identifier tokens
METHOD_IDENTIFIER: Method names (like HEADWIND, CROSSWIND, RUNWAYHEADING, WIND)
AIRCRAFT_ID : Aircraft registration (like C-GNBL)
FLIGHT_ID : Flight numbers (like AL123)
INTEGER : Integer literals
FLOAT : Float literals
DECIMAL : Decimal point token
STRING_LITERAL : String literals in quotes
DATE_LITERAL : Date format '2025-05-15'
BOOLEAN : TRUE/FALSE values
COMMENT_LITERAL : Comments starting with ^^

Operators:

EQL_T : =
COLON_T : :
NOT_EQ_T : !=
GT_T : >
PLUS_T : +
MULTI_T : *
DIV_T : /
COMMA_T : ,
DEC_T : .

Delimiters:

LBR_T : {
RBR_T : }
LPR_T : (
RPR_T :)
EOS_T : ; (end of statement)

Keywords:

Program Structure: MAIN, ENDMAIN, BRIEFING, ENDBRIEFING, DISPATCH, ENDDISPATCH, LOADSHEET, ENDLOADSHEET, WEATHER, ENDWEATHER

Data Blocks: AIRCRAFT, FLIGHT, ROUTE, RECEIVEDDATA, ENDRECEIVEDDATA, RUNWAYDATA, ENDRUNWAYDATA, WINDANALYSIS, ENDWINDANALYSIS

Control Flow: IF, THEN, ELSE, ENDIF, PRINT, REPORT, ENDREPORT

Aviation Functions: AIRPATH, TOTALWEIGHT, WEIGHTBAL, VALIDATEWB, HEADWIND, CROSSWIND

Aircraft Types: CESSNA172, PA28, B747-400

Weather Data: METAR, NOTAM, SAFETYALERT, ENDSAFETYALERT

Boolean Values: true, false

Loading Operations: LOAD_AIRCRAFT, LOAD_PILOT, LOAD_PASSENGER, LOAD_FUEL_FULL, LOAD_FUEL_TABS, LOAD_BAGGAGE, WB_CHECK

Integration: REQUEST, FROM, WITHCONFIG, CALCULATE

AirLang's complete keyword set encompasses program structure keywords, aviation-specific identifiers, and specialized function names. The full keyword table is maintained in the scanner implementation and available in the implementation repository.

APPENDIX C

VIRTUAL MACHINE INSTRUCTION SET

Core Architecture

Stack-Based Virtual Machine

- Maximum stack size: Configurable
- Value types: VAL_NUMBER (double), VAL_STRING (fixed-length)
- Variable storage: MAX_VARIABLES with name-value pairs
- Instruction format: OpCode + Operand (union of string/number)

Instruction Categories (237+ Instructions Total)

1. Core Stack Operations

OP_LOAD_NUM	Load numeric literal onto stack
OP_LOAD_STR	Load string literal onto stack
OP_LOAD_FUNCTION	Load function reference for execution
OP_STORE_VAR	Store stack value to variable
OP_HALT	Terminate execution

2. Aviation-Specific Data Operations

OP_STORE_AIRCRAFT_ID	Store with aircraft ID validation
OP_STORE_COORDS	Store with coordinate format validation
OP_STORE_DATE	Store with date format validation

3. Mathematical and Expression Processing

OP_CALC_EXPRESSION	Evaluate mathematical expressions
OP_CALC_DISTANCE	Calculate great circle distance (Haversine)

4. Aviation Domain Calculations

OP_CALC_HEADWIND	Calculate headwind component
OP_CALC_CROSSWIND	Calculate crosswind component
OP_CALC_TOTALWEIGHT	Calculate aircraft total weight
OP_CALC_MOMENT	Calculate weight & balance moment
OP_CALC_CG	Calculate center of gravity position
OP_CALC_VALIDATEWB	Validate weight & balance limits

5. Control Flow Instructions

OP_IF	Begin conditional block
OP_ELSE	Alternative conditional path
OP_ENDIF	End conditional block
OP_CONDITION	Evaluate conditional expression

6. Output Operations

OP_PRINT	Output simple values
OP_PRINT_INTERPOLATED	Output formatted expressions with variables

7. Program Structure Management

The VM uses structured opcodes to manage AirLang's hierarchical block syntax:

Primary Blocks:

OP_ENTER_MAIN / OP_EXIT_MAIN	- Program boundary
------------------------------	--------------------

OP_ENTER_BRIEFING / OP_EXIT_BRIEFING	- Flight briefing section
OP_ENTER_WEATHER / OP_EXIT_WEATHER	- Weather analysis section
OP_ENTER_LOADSHEET / OP_EXIT_LOADSHEET	- Weight calculations
OP_ENTER_DISPATCH / OP_EXIT_DISPATCH	- Final authorization

Sub-Blocks:

OP_ENTER_AIRCRAFT	- Aircraft specification
OP_ENTER_FLIGHT	- Flight information
OP_ENTER_ROUTE	- Route definition
OP_ENTER_RECEIVEDDATA	- METAR weather data
OP_ENTER_RUNWAYDATA	- Airport runway config
OP_ENTER_WINDANALYSIS	- Wind component analysis
OP_ENTER_SAFETYALERT	- Safety threshold checking
OP_ENTER_REPORT	- Report generation

Instruction Format Specification

Instruction Structure:

```
typedef struct {
    int opCode;
    union {
        double num_operand;
        char str_operand[MAX_STRING_LENGTH];
    } operand;
} Instruction;
```

Execution Model

Virtual Machine State:

- **Program Counter (PC):** Current instruction index
- **Stack Pointer (SP):** Current stack position
- **Variable Storage:** Named variable table with type information
- **Condition State:** Boolean result for control flow
- **Safety Context:** Current aircraft/airport for calculations

Error Handling:

- Stack overflow/underflow detection
- Aviation constraint validation
- Type safety enforcement
- Runtime safety limit checking
- Aviation-Specific Features

Safety Validation:

- Aircraft weight limits (MTOW checking)
- Center of gravity boundaries
- Fuel capacity constraints
- Weather condition thresholds

Domain Integration:

- METAR parsing with automatic airport context
- Wind triangle mathematics
- Great circle navigation calculations
- Weight & balance computations

This instruction set provides complete support for aviation flight planning operations while maintaining the safety-critical validation requirements essential for aviation software systems.

APPENDIX D

COMPLETE SAMPLE PROGRAM WITH OUTPUT

D.1 Comprehensive Flight Planning Example

This sample demonstrates AirLang's integrated approach to flight planning, combining aircraft specifications, weather analysis, weight-and-balance calculations, and dispatch authorization in a single executable program. The code showcases domain-specific syntax for aviation identifiers, automatic safety validation, and specialized mathematical functions.

```
%AirLang Enhanced Flight Plan - Demonstrates comprehensive flight dispatch with detailed
W&B reporting. Shows aircraft specs, weather evaluation, enhanced weight & balance reporting, fuel
planning, and dispatch authorization%
```

```
REQUEST METAR FROM "https://airlangMetar.fly" ;
REQUEST NOTAM FROM "https://airlangNotam.fly" ;

MAIN {
    BRIEFING {
        ^^ Aircraft Configuration
        AIRCRAFT {
            AircraftID: C-GNBL;
            AircraftType: "Piper Cherokee PA-28-140";
            MaxTakeoffWeight: 2150;
            EmptyWeight: 1410;
            FuelCapacity: 300;
            CruiseSpeed: 108;
            FuelBurnRate: 8;
        }

        FLIGHT {
            ^^ Flight Information
            FlightNumber: AL123;
            FlightDate: '2025-05-15';
            ScheduledDeparture: "14:30Z";
            ScheduledArrival: "16:00Z";
            WeatherCondition: "RAIN";
            FlightLevel: "3500";
            EstimatedFlightTime: 90;
            EstimatedFuelRequired: 120;
            ReserveFuel: 60;
        }

        ROUTE {
            ^^ Route Planning
            DepartureAirport: "CYOW";
            DepartureName: "Ottawa Macdonald-Cartier International";
            DepartureCoords: 45.3225, -75.6692;
            ArrivalAirport: "CYYZ";
            ArrivalName: "Toronto Pearson International";
            ArrivalCoords: 43.6777, -79.6248;
            AlternateAirport: "CYTZ";
        }
    } ENDBRIEFING;

    WEATHER {
        RECEIVEDDATA {
            METAR: "METAR CYOW 251630Z 27015G25KT 5000 -RA BKN008 OVC015 08/06 A2995 RMK";
            METAR: "METAR CYYZ 251630Z 24012KT 6SM -RA SCT010 BKN020 09/07 A2998 RMK";
        } ENDRECEIVEDDATA;

        RUNWAYDATA {
            RUNWAYHEADING() WITHCONFIG {
                CYOW_RUNWAY: 070;
            }
        }
    }
}
```

```

        CYYZ_RUNWAY: 230;
    };
} ENDRUNWAYDATA;

WINDANALYSIS {
    ^^ Using (wind_dir, wind_speed, runway_heading) from METAR data

    WIND() WITHCONFIG {
        CYOW = HEADWIND();
        CYOW = CROSSWIND();
        CYYZ = HEADWIND();
        CYYZ = CROSSWIND();
    };
} ENDWINDANALYSIS;

SAFETYALERT{
    IF CYOW_WIND_GUST > 20 THEN
        PRINT{"ALERT: Excessive gusts at CYOW - " + CYOW_WIND_GUST + "kt (Limit: 20kt)"};
    ENDIF;

    IF CYOW_CROSSWIND > 15 THEN
        PRINT{"WARNING: High crosswind at CYOW - " + CYOW_CROSSWIND + "kt (Limit: 15kt)"};
        PRINT { "NOTE: Time to show off those crosswind landing skills" };
    ENDIF;

    IF CYYZ_WIND_GUST > 20 THEN
        PRINT { "ALERT: Excessive gusts at CYYZ - " + CYYZ_WIND_GUST + "kt" };
    ENDIF;

    IF CYYZ_CROSSWIND > 15 THEN
        PRINT { "WARNING: High crosswind at CYYZ - " + CYYZ_CROSSWIND + "kt" };
    ENDIF;
} ENDSAFETYALERT;
} ENDWEATHER;

LOADSHEET {
    ^^ Load Sheet Calculations
    PassengerCount: 3;
    CargoWeight: 10;

    PassengerWeight = PassengerCount * 170;
    PayloadWeight = PassengerWeight + CargoWeight;
    TotalWeight = EmptyWeight + PayloadWeight + EstimatedFuelRequired;

    ^^ Performance Calculations
    Headwind: 30;
    GroundSpeed = CruiseSpeed + (-30);
    Distance = AIRPATH;
    FlightTimeHours = Distance / GroundSpeed;
    TotalFuelBurn = FlightTimeHours * FuelBurnRate;
    TotalFuelRequired = TotalFuelBurn + ReserveFuel;

    ^^ Enhanced Weight and Balance Calculations
    AircraftTotalWeight = TOTALWEIGHT;
    TotalMoment = WEIGHTBAL;
    CenterOfGravity = CENTEROFGRAVITY;
    WeightBalanceStatus = VALIDATEWB;

    ^^ Calculate margins for display
    WeightMargin = MaxTakeoffWeight - AircraftTotalWeight;
    FuelMargin = FuelCapacity - TotalFuelRequired;
    OverweightBy = AircraftTotalWeight - MaxTakeoffWeight;

    ^^ Weight and Balance Safety Checks

```

```

IF AircraftTotalWeight > MaxTakeoffWeight THEN
    PRINT { "WARNING: Aircraft exceeds Maximum Takeoff Weight" };
    PRINT { "Calculated Weight: " + AircraftTotalWeight + " lbs" };
    PRINT { "MTOW Limit: " + MaxTakeoffWeight + " lbs" };
    PRINT { "Overweight by: " + OverweightBy + " lbs" };
ELSE
    PRINT { "Weight Check: WITHIN LIMITS" };
    PRINT { "Calculated Weight: " + AircraftTotalWeight + " lbs" };
    PRINT { "Available Margin: " + WeightMargin + " lbs" };
ENDIF;

^^ Center of Gravity Validation
IF WeightBalanceStatus != 1 THEN
    PRINT { "WARNING: Weight & Balance OUT OF LIMITS" };
    PRINT { "Total Moment: " + TotalMoment + " lb-in" };
    PRINT { "CG Position: " + CenterOfGravity + " inches from datum" };
    PRINT { "CG Position may be unsafe - Check loading!" };
ELSE
    PRINT { "Weight & Balance: WITHIN LIMITS" };
    PRINT { "Total Moment: " + TotalMoment + " lb-in" };
    PRINT { "CG Position: " + CenterOfGravity + " inches from datum" };
    PRINT { "CG Position: SAFE" };
ENDIF;

^^ Fuel Capacity Validation
IF TotalFuelRequired > FuelCapacity THEN
    PRINT { "WARNING: Insufficient fuel capacity" };
    PRINT { "Required Fuel: " + TotalFuelRequired + " lbs" };
    PRINT { "Fuel Capacity: " + FuelCapacity + " lbs" };
ELSE
    PRINT { "Fuel Check: ADEQUATE CAPACITY" };
    PRINT { "Required Fuel: " + TotalFuelRequired + " lbs" };
    PRINT { "Remaining Capacity: " + FuelMargin + " lbs" };
ENDIF;

} ENLOADSHEET;

DISPATCH {
    ^^ Weather Dispatch Decision
    IF FuelBurnRate > FuelCapacity THEN
        PRINT { "NOTAM: Weather conditions may affect departure" };
        PRINT { "Current Weather: " + WeatherCondition };
        PRINT { "Monitor conditions and standby for updates" };
    ELSE
        PRINT { "NOTAM: Clear weather - Flight ON TIME" };
    ENDIF;

    REPORT {
        ^^ Flight Summary Report
        PRINT { "" };
        PRINT { "===== " };
        PRINT { "                AIRLANG FLIGHT DISPATCH SUMMARY                " };
        PRINT { "===== " };
        PRINT { "Flight: " + FlightNumber + " on " + FlightDate };
        PRINT { "Aircraft: " + AircraftID + " (" + AircraftType + ")" };
        PRINT { "Route: " + DepartureAirport + " to " + ArrivalAirport };
        PRINT { "Distance: " + Distance + " nautical miles" };
        PRINT { "Estimated Flight Time: " + FlightTimeHours + " hours" };
        PRINT { "Ground Speed: " + GroundSpeed + " knots" };
        PRINT { "Flight Level: " + FlightLevel };
        PRINT { "Weather: " + WeatherCondition };
        PRINT { "Passenger Load: " + PassengerCount + " souls on board" };
        PRINT { "===== " };

        ^^ Weather Report
    }
}

```

```

PRINT { "" };
PRINT { "===== WEATHER REPORT =====" };
PRINT { "Departure Airport (CYOW):" };
PRINT { "  - Headwind: " + CYOW_HEADWIND + "kt" };
PRINT { "  - Crosswind: " + CYOW_CROSSWIND + "kt" };
PRINT { "  - Runway in use: 07 (heading 070)" };
PRINT { "" };
PRINT { "Arrival Airport (CYYZ):" };
PRINT { "  - Headwind: " + CYYZ_HEADWIND + "kt" };
PRINT { "  - Crosswind: " + CYYZ_CROSSWIND + "kt" };
PRINT { "  - Runway in use: 23 (heading 230)" };
PRINT { "" };
PRINT { "===== " };

^^ Enhanced Weight & Balance Report
PRINT { "" };
PRINT { "===== WEIGHT & BALANCE REPORT =====" };
PRINT { "Aircraft Configuration: PA28" };
PRINT { "Empty Weight: " + EmptyWeight + " lbs" };
PRINT { "Payload Weight: " + PayloadWeight + " lbs" };
PRINT { "Fuel Load: " + EstimatedFuelRequired + " lbs" };
PRINT { "Total Weight: " + AircraftTotalWeight + " lbs" };
PRINT { "MTOW Limit: " + MaxTakeoffWeight + " lbs" };
PRINT { "" };
PRINT { "Center of Gravity Analysis:" };
PRINT { "  CG Position: " + CenterOfGravity + " inches from datum" };
PRINT { "  Total Moment: " + TotalMoment + " lb-in" };
PRINT { "" };
PRINT { "Loading Breakdown:" };
PRINT { "  Passengers: " + PassengerCount + " persons" };
PRINT { "  Passenger Weight: " + PassengerWeight + " lbs" };
PRINT { "  Baggage Weight: " + CargoWeight + " lbs" };
PRINT { "  Fuel Weight: " + EstimatedFuelRequired + " lbs" };
PRINT { "  Reserve Fuel: " + ReserveFuel + " lbs" };
PRINT { "===== " };

^^ Final Dispatch Authorization
PRINT { "" };
PRINT { "===== DISPATCH AUTHORIZATION =====" };
PRINT { " All pre-flight requirements satisfied" };
PRINT { " Aircraft airworthy and within limits" };
PRINT { " Weather conditions acceptable" };
PRINT { " Fuel planning adequate with reserves" };
PRINT { " Weight & Balance within certified envelope" };
PRINT { "" };
PRINT { "*** DISPATCH STATUS: CLEARED FOR DEPARTURE ***" };
PRINT { "Captain authorization required for pushback" };
PRINT { "Contact Ground Control on 121.9 for taxi clearance" };
PRINT { "Safe flight - " + FlightNumber };
PRINT { "===== " };
PRINT { "" };
} ENDREPORT;
} ENDDISPATCH;
} ENDMAIN;

```

D.2 Expected Program Output

When executed, this program produces the following comprehensive flight dispatch report:

AIRCRAFT DATABASE: Loaded configuration for PA28 (Piper Cherokee PA-28-140)

AIRLANG FLIGHT DISPATCH SUMMARY

Flight: AL123 on 2025-05-15
Aircraft: C-GNBL (Piper Cherokee PA-28-140)
Route: CYOW to CYYZ
Distance: 196.10 nautical miles
Estimated Flight Time: 2.51 hours
Ground Speed: 78 knots
Flight Level: 3500
Weather: RAIN
Passenger Load: 3 souls on board

WEATHER REPORT

Departure Airport (CYOW):
- Headwind: -15.36 kt
- Crosswind: 4.73 kt
- Runway in use: 07 (heading 070)

Arrival Airport (CYYZ):
- Headwind: 11.82 kt
- Crosswind: 2.08 kt
- Runway in use: 23 (heading 230)

WEIGHT & BALANCE REPORT

Aircraft Configuration: PA28
Empty Weight: 1410 lbs
Payload Weight: 520 lbs
Fuel Load: 120 lbs
Total Weight: 2040 lbs
MTOW Limit: 2150 lbs

Center of Gravity Analysis:
CG Position: 93.84 inches from datum
Total Moment: 191430 lb-in

Loading Breakdown:
Passengers: 3 persons
Passenger Weight: 510 lbs
Baggage Weight: 10 lbs
Fuel Weight: 120 lbs
Reserve Fuel: 60 lbs

DISPATCH AUTHORIZATION

All pre-flight requirements satisfied
Aircraft airworthy and within limits
Weather conditions acceptable
Fuel planning adequate with reserves

Weight & Balance within certified envelope

*** DISPATCH STATUS: CLEARED FOR DEPARTURE ***

Captain authorization required for pushback

Contact Ground Control on 121.9 for taxi clearance

Safe flight - AL123

PROJECT REPOSITORY

The complete AirLang implementation, including compiler source code, documentation, and example programs, is available at:

<https://github.com/tishap27/AirLang>